Case Study Report

13. Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Submitted by: **PRAJESH P**
Register No.: **241401067**

241401067

Note: The enterprise scenario in this report is an illustrative composite based on commonly documented low-code adoption challenges. It is not a record of a specific named company.

Table of Contents

1. Abstract

2. Introduction

3. Background: What Is Low-Code?

4. Case Scenario: Global Insurance Claims Modernization

5. Limitations Encountered

6. Impact Analysis

7. Comparison: Low-Code vs. Traditional Development

8. Lessons Learned and Recommendations

9. When Low-Code Is Still the Right Choice

10. Conclusion

11. References

241401067

1. Abstract

Low-code development platforms (LCDPs) such as OutSystems, Mendix, Microsoft Power Apps and Appian promise faster delivery of business applications through visual modelling, prebuilt components and minimal hand-written code. They have proven valuable for departmental tools, workflow automation and prototypes. However, when applied to complex, large-scale enterprise systems, their limitations become significant. This case study examines an illustrative scenario in which a large insurer attempts to rebuild its core claims-processing system on a low-code platform. It identifies the technical, architectural, financial and organisational limitations that emerged, analyses their impact, and offers recommendations for deciding when and how low-code should be used in enterprise settings.

2. Introduction

Enterprises face constant pressure to deliver digital solutions faster while IT talent remains scarce. Industry analysts have repeatedly forecast that low-code and no-code approaches will account for a growing share of new application development. The appeal is clear: shorter time-to-market, lower initial cost, and the ability to involve business users (citizen developers) directly.

Yet speed in the early stages does not always translate into long-term success. Complex enterprise use cases involve high transaction volumes, intricate business rules, legacy integration, strict regulatory compliance and long application lifecycles. These conditions test the boundaries of any abstraction layer. The objective of this report is to critically assess where low-code platforms fall short under such conditions.

3. Background: What Is Low-Code?

A low-code platform provides a visual development environment in which applications are assembled using drag-and-drop interfaces, model-driven logic, configurable connectors and reusable templates. Custom code can usually be added for exceptions, but the platform's runtime, data layer and deployment pipeline are largely managed by the vendor.

Typical strengths include rapid prototyping, built-in security and authentication features, simplified deployment, and reduced dependence on specialised programmers. These strengths are exactly what make low-code attractive, and the same design choices (abstraction and vendor-managed infrastructure) are the source of its limitations at scale.

4. Case Scenario: Global Insurance Claims Modernization

4.1 Organisation and Objective

GlobalShield Insurance (a fictional name used for illustration) operates in 14 countries and processes roughly 2.5 million claims per year. Its core claims system, built over 20 years on mainframe and Java components, was slow to change. Leadership selected a leading low-code

241401067

platform to rebuild claims intake, adjudication, fraud checks, payments and customer communication within 18 months, targeting a 40% reduction in delivery time and cost compared with a conventional rewrite.

4.2 Complexity Factors

- **Volume:** peaks of over 50,000 claim transactions per hour during catastrophic events such as floods.
- **Business rules:** thousands of country-specific rules for coverage, deductibles, and fraud scoring.
- **Integration:** 30+ systems including a mainframe policy ledger, payment gateways, a machine-learning fraud engine, and regulator reporting portals.
- **Compliance:** GDPR, local insurance regulations, audit trails and data residency requirements.
- **Lifespan:** an expected system life of 10 to 15 years with continuous change.

4.3 Project Outcome

The first modules (customer portal and simple claim intake) were delivered in four months, well ahead of plan, and stakeholders were enthusiastic. As the project moved into adjudication logic, fraud integration and high-volume processing, progress slowed sharply. By month 14, the team had built extensive custom code and external microservices to work around platform constraints. The programme was eventually restructured into a hybrid architecture, with low-code retained only for the front-end and workflow layers. The overall cost exceeded the original conventional-development estimate.

5. Limitations Encountered

5.1 Scalability and Performance

Under load testing, the platform's generated runtime could not sustain the required transaction throughput. Auto-generated database queries were not optimal, and the team had limited ability to tune indexes, caching or query plans. Scaling depended on the vendor's licensing tiers and infrastructure rather than on architecture choices the team could control. Performance problems that a traditional team would solve by refactoring code became support tickets and paid upgrades.

5.2 Handling Complex Business Logic

Visual logic builders work well for linear workflows but become unwieldy with deeply nested conditions. The claims rules, with thousands of branching conditions, produced sprawling diagrams that were hard to read, review, test and version. Developers described them as 'spaghetti in pictures'. Automated unit testing and code-review practices, standard in conventional development, were difficult to apply to visual models.

5.3 Integration with Legacy and Third-Party Systems

241401067

Prebuilt connectors covered common SaaS products, but the mainframe ledger and the custom fraud engine required bespoke adapters. Complex protocols, batch interfaces, message queues and non-standard authentication demanded custom extensions written in the platform's supported languages. These extensions were poorly documented and tightly coupled to the platform version, which created fragile integrations.

5.4 Vendor Lock-In

Applications built on a proprietary platform are modelled in a vendor-specific format and run on a vendor-specific runtime. Source code that can be exported is often not human-readable or portable. GlobalShield therefore faced high switching costs: leaving the platform would effectively mean a complete rewrite. This weakened its negotiating position on renewal pricing and tied its roadmap to the vendor's product decisions.

5.5 Customisation and Flexibility Constraints

When requirements fell outside the platform's component library, workarounds were needed. Custom UI behaviour, specialised algorithms, and fine-grained control over concurrency and memory were difficult or impossible. Each workaround added technical debt, partially defeating the purpose of low-code.

5.6 Security, Compliance and Data Residency

Although platforms provide baseline security, the enterprise could not fully inspect the underlying code or infrastructure. Auditors asked for evidence about encryption handling, patching cadence and data location that depended on vendor documentation. Some country-specific data residency needs required costly private-cloud deployment options. Misconfigured access rules by less-experienced citizen developers also created governance risks.

5.7 Governance, Shadow IT and Application Sprawl

Easy app creation led business units to build overlapping tools without central oversight. Duplicate data models, inconsistent security settings and unmanaged dependencies appeared across the portfolio. Without strong governance, standards and ownership, maintenance costs increased over time.

5.8 Cost at Scale

Licensing is commonly based on users, applications, or resource consumption. Costs that looked modest during the pilot grew steeply with external customers, data volumes and additional environments. A total-cost-of-ownership review at month 12 showed that licensing, specialist consultants and workaround development eroded the projected savings.

5.9 Skills, Testing and DevOps Maturity

Skilled low-code developers for this specific platform were scarce and expensive. Integrating the platform with the enterprise CI/CD pipeline, automated regression testing, and source-control branching strategies was more constrained than with conventional tools. The perceived 'no need for

241401067

developers' promise did not hold for complex work, since professional engineering skills were still essential.

5.10 Summary of Limitations

Limitation	Observed Symptom in the Case	Severity
Scalability	Throughput shortfall under peak claim loads	High
Complex logic	Unreadable models, weak testing	High
Legacy integration	Custom adapters, fragile interfaces	High
Vendor lock-in	Costly exit, pricing leverage lost	High
Customisation	Workarounds and technical debt	Medium
Security/compliance	Audit evidence dependent on vendor	Medium
Governance	App sprawl, inconsistent standards	Medium
Cost at scale	Licensing growth beyond forecast	High
Skills/DevOps	Scarce talent, pipeline constraints	Medium

6. Impact Analysis

The limitations above had measurable consequences for the programme. The table below presents an illustrative comparison between the original plan and the realised outcome.

Metric	Planned	Realised (Illustrative)
Delivery time	18 months	28 months
Budget versus conventional rewrite	40% lower	About 15% higher
Peak throughput supported (claims/hour)	50,000	Approx. 18,000 before redesign
Share of system on low-code	100%	About 45% (front-end and workflow)
Custom code / external services	Minimal	Significant (core rules and integration)

Beyond numbers, the organisation experienced reduced stakeholder confidence, delayed regulatory commitments and team fatigue from repeated redesign. These figures are illustrative and are provided to show how limitations compound, not to represent measured data from a real company.

7. Comparison: Low-Code vs. Traditional Development

Aspect	Low-Code Platform	Traditional Development
Initial speed	Very fast for standard apps	Slower start

Aspect	Low-Code Platform	Traditional Development
Complex logic	Difficult to manage visually	Flexible, testable code
Scalability control	Limited by vendor runtime	Full architectural control
Integration depth	Strong for common connectors only	Unlimited with custom code
Portability	Low (lock-in)	High (open standards)
Cost profile	Low upfront, rising licences	Higher upfront, predictable
Skill requirement	Platform-specific	General engineering
Best fit	Workflows, forms, prototypes	Core systems, high performance

8. Lessons Learned and Recommendations

- **Assess fit before committing.** Evaluate transaction volume, logic complexity and integration needs against platform capabilities through a proof of concept that tests the hardest requirements first, not the easiest.
- **Adopt a hybrid architecture.** Use low-code for user interfaces, workflows and rapid-change layers, and keep performance-critical and rule-heavy cores in conventional, well-tested services exposed through APIs.
- **Design for exit.** Keep business logic and data models as portable as possible, document them outside the platform, and negotiate data and code export rights in contracts.
- **Model total cost of ownership.** Include licences at projected scale, environments, consultants, and workaround development over a 5 to 10 year horizon.
- **Establish governance early.** Create a centre of excellence, naming and security standards, approval processes for new applications and clear ownership.
- **Keep professional engineering practices.** Apply version control, automated testing, code review and CI/CD to low-code projects as far as the platform allows.
- **Verify security and compliance evidence.** Obtain vendor certifications, penetration test reports and data residency guarantees before design decisions are final.

9. When Low-Code Is Still the Right Choice

The limitations above do not make low-code unsuitable for enterprises. It remains highly effective for internal approval workflows, data-collection forms, dashboards, customer self-service portals, process automation, proof-of-concept builds and applications with a short expected lifespan or modest scale. The key is matching the tool to the problem: the mistake in the case was not choosing low-code, but expecting it to carry the most demanding core workload of the business.

241401067

10. Conclusion

Low-code platforms deliver real value by accelerating development and widening participation, but their abstraction comes at a price. For complex enterprise use cases characterised by high scale, intricate logic, heavy legacy integration and stringent compliance, the limitations in scalability, flexibility, portability, cost control and governance can outweigh the early speed advantage. The illustrative case of GlobalShield shows that the most successful path is a deliberate, hybrid strategy: use low-code where it excels, retain conventional engineering where control matters, and plan for governance and exit from the beginning. Enterprises that approach low-code as one tool among many, rather than a universal replacement for software engineering, are best placed to benefit from it.

11. References

- Gartner, Inc. Research publications on enterprise low-code application platforms (Magic Quadrant series).

- Forrester Research. Reports on low-code development platforms and the future of application development.

- Bock, A. C., and Frank, U. (2021). Low-Code Platform. Business & Information Systems Engineering, 63(6), 733-740.

- Sahay, A., Indamutsa, A., Di Ruscio, D., and Pierantonio, A. (2020). Supporting the understanding and comparison of low-code development platforms. 46th Euromicro Conference on Software Engineering and Advanced Applications.

- Vendor documentation: OutSystems, Mendix, Microsoft Power Apps and Appian official platform documentation.

241401067